

DART PROGRAMMING

A Complete Beginner's Guide — Part 2

Loops • Strings • Booleans • Lists • Maps • Functions • Interfaces • Enums • Runes

Chapter 7: Loops — Repeating Instructions

At times, certain instructions require repeated execution. Loops are the ideal way to handle this. A loop represents a set of instructions that must be repeated. In a loop's context, each repetition is called an iteration.

7.1 Classification of Loops

Loops in Dart are classified into two main categories:

- Definite Loops — the number of iterations is fixed and known in advance.
- Indefinite Loops — the number of iterations is unknown and depends on a condition.

7.2 Definite Loops

The for Loop

The for loop executes a code block for a specified number of times. It is used to iterate over a fixed set of values.

```
void main() {  
  for (var i = 0; i < 5; i++) {  
    print('Iteration: ${i}');  
  }  
}
```

Output:

```
Iteration: 0  
Iteration: 1  
Iteration: 2  
Iteration: 3  
Iteration: 4
```

The for...in Loop

The for...in loop iterates over the elements of a collection (like a List or Map keys).

```
void main() {  
  var fruits = ['apple', 'banana', 'mango'];  
  for (var fruit in fruits) {  
    print(fruit);  
  }  
}
```

Output:

```
apple  
banana  
mango
```

7.3 Indefinite Loops

The while Loop

The while loop evaluates a condition before executing the block. It keeps running as long as the condition is true.

```
void main() {  
    var i = 0;  
    while (i < 3) {  
        print('Count: ${i}');  
        i++;  
    }  
}
```

Output:

```
Count: 0  
Count: 1  
Count: 2
```

The do...while Loop

The do...while loop executes the block first and then checks the condition. This means the block always runs at least once.

```
void main() {  
    var i = 0;  
    do {  
        print('Value: ${i}');  
        i++;  
    } while (i < 3);  
}
```

Output:

```
Value: 0  
Value: 1  
Value: 2
```

7.4 Loop Control Statements

break Statement

The break statement exits the loop immediately, regardless of whether the condition is still true.

```
void main() {  
    for (var i = 0; i < 5; i++) {  
        if (i == 3) break;  
        print(i);  
    }  
}
```

Output:

```
0  
1  
2
```

continue Statement

The continue statement skips the rest of the current iteration and jumps back to the loop condition.

```
void main() {  
    for (var i = 0; i < 5; i++) {  
        if (i == 2) continue;  
        print(i);  
    }  
}
```

Output:

```
0  
1  
3  
4
```

7.5 Using Labels to Control Flow

A label is an identifier followed by a colon (:) applied to a loop. Labels let you use break and continue on outer loops from inside nested loops.

☐ **Note:** Line breaks are NOT allowed between the break/continue statement and its label name.

Example: Label with break

```
void main() {  
    outerloop:  
    for (var i = 0; i < 5; i++) {  
        print('Outerloop: ${i}');  
        innerloop:  
        for (var j = 0; j < 5; j++) {  
            if (j > 3) break;  
            if (i == 2) break innerloop;  
            if (i == 4) break outerloop;  
            print('Innerloop: ${j}');  
        }  
    }  
}
```

Example: Label with continue

```
void main() {  
    outerloop:  
    for (var i = 0; i < 3; i++) {  
        print('Outerloop:${i}');  
        for (var j = 0; j < 5; j++) {  
            if (j == 3) continue outerloop;  
            print('Innerloop:${j}');  
        }  
    }  
}
```

Output:

```
Outerloop: 0  
Innerloop: 0
```

```
Innerloop: 1
Innerloop: 2
Outerloop: 1
Innerloop: 0
Innerloop: 1
Innerloop: 2
Outerloop: 2
Innerloop: 0
Innerloop: 1
Innerloop: 2
```

Chapter 8: Strings — Working with Text

The String data type represents a sequence of characters. A Dart string is a sequence of UTF-16 code units. Strings are immutable in Dart — once created they cannot change, but operations can produce new string values.

8.1 Declaring Strings

String values can be represented using single quotes, double quotes, or triple quotes for multi-line strings.

```
String str1 = 'single line with single quotes';
String str2 = "single line with double quotes";
String str3 = '''
    multi-line
    string here
''';
String str4 = """
    another multi-line
    string here
""";
```

8.2 String Interpolation

Interpolation is the process of embedding values inside a string. Dart uses the `{}` syntax for this.

```
void main() {
    String str1 = 'hello';
    String str2 = 'world';
    String res = str1 + str2;
    print('The concatenated string: ${res}');

    int n = 1 + 1;
    print('The sum of 1 and 1 is ${n}');
    print('The sum of 2 and 2 is ${2+2}');
}
```

Output:

```
The concatenated string: helloworld
The sum of 1 and 1 is 2
The sum of 2 and 2 is 4
```

8.3 String Properties

Property	Description
<code>codeUnits</code>	Returns an unmodifiable list of the UTF-16 code units of this string.
<code>isEmpty</code>	Returns true if this string is empty.
<code>length</code>	Returns the length of the string including space, tab, and newline characters.

8.4 String Methods

Method	Description
toLowerCase()	Converts all characters to lower case.
toUpperCase()	Converts all characters to upper case.
trim()	Removes leading and trailing whitespace.
compareTo()	Compares this object to another.
replaceAll()	Replaces all substrings matching a pattern with a given value.
split()	Splits the string at matches of a delimiter and returns a list.
substring()	Returns substring from startIndex (inclusive) to endIndex (exclusive).
toString()	Returns a string representation of this object.
codeUnitAt()	Returns the 16-bit UTF-16 code unit at the given index.

Chapter 9: Booleans — true and false

Dart provides built-in support for the Boolean data type. A Boolean can only hold one of two values: true or false. The keyword `bool` is used to declare a Boolean variable.

9.1 Declaring Booleans

```
bool var_name = true;  
// OR  
bool var_name = false;
```

9.2 Boolean Example

```
void main() {  
  bool test;  
  test = 12 > 5;  
  print(test);  
}
```

Output:

```
true
```

9.3 Dart vs JavaScript: Boolean Strictness

Unlike JavaScript, Dart's Boolean type only accepts the literal value `true` as truthy. Any other value — including a non-empty string — is treated as false.

```
void main() {  
  var str = 'abc';  
  if (str) {  
    print('String is not empty');  
  } else {  
    print('Empty String');  
  }  
}
```

Output:

```
Empty String // In unchecked/production mode
```

☐ **Note:** In checked mode, using a non-bool value in an if condition throws an exception: type 'String' is not a subtype of type 'bool'. Always use checked mode while learning!

Chapter 10: Lists — Ordered Collections

Dart represents arrays in the form of List objects. A List is simply an ordered group of objects. Each element in a List is identified by a zero-based index. The dart:core library provides the List class for creating and manipulating lists.

10.1 Fixed Length List

A fixed-length list cannot grow or shrink at runtime. Any attempt to resize it will result in an exception.

```
void main() {  
  final fixedList = List<int>.filled(5, 0, growable: false);  
  print(fixedList);  
}
```

Output:

```
[0, 0, 0, 0, 0]
```

10.2 Growable List

A growable list can change in size at runtime. You can declare it with initial values or as an empty list and add elements dynamically.

```
// Declare with values  
var num_list = [1, 2, 3];  
  
// Declare empty, then add  
var lst = [];  
lst.add(12);  
lst.add(13);  
print(lst);
```

Output:

```
[12, 13]
```

10.3 List Properties

Property	Description
first	Returns the first element in the list.
isEmpty	Returns true if the collection has no elements.
isNotEmpty	Returns true if the collection has at least one element.
length	Returns the size of the list.
last	Returns the last element in the list.
reversed	Returns an iterable with the list's values in reverse order.
single	Checks if the list has only one element and returns it.

10.4 List Operations

Inserting Elements

Use `add()` to append a single element, or `addAll()` to append multiple elements.

```
var list = [1, 2, 3];
list.add(4);
print(list);    // [1, 2, 3, 4]
```

Updating Elements

Update by index assignment, or use `replaceRange()` to replace a range of elements.

```
var list = [10, 20, 30];
list[1] = 99;
print(list);    // [10, 99, 30]

list.replaceRange(0, 2, [5, 6]);
print(list);    // [5, 6, 30]
```

Removing Elements

Use `remove()` to remove a specific value, or `removeAt()` to remove by index.

```
var list = [1, 2, 3, 4];
list.remove(3);
print(list);    // [1, 2, 4]

list.removeAt(0);
print(list);    // [2, 4]
```

Chapter 11: Maps — Key-Value Pairs

A Map stores data as key-value pairs. Keys and values can be of any type. Maps are dynamic — they can grow and shrink at runtime. Think of a Map like a real-world dictionary where each word (key) maps to a definition (value).

11.1 Declaring a Map using Map Literals

Enclose key-value pairs inside curly brackets { }.

```
var identifier = { key1: value1, key2: value2 };
void main() {
    var details = {'Username': 'tom', 'Password': 'pass@123'};
    print(details);
}
```

Output:

```
{Username: tom, Password: pass@123}
```

11.2 Declaring a Map using the Map Constructor

```
void main() {
    var details = new Map();
    details['Username'] = 'admin';
    details['Password'] = 'admin@123';
    print(details);
}
```

Output:

```
{Username: admin, Password: admin@123}
```

11.3 Adding Values to a Map at Runtime

```
void main() {
    var details = {'Username': 'tom', 'Password': 'pass@123'};
    details['Uid'] = 'U1001';
    print(details);
}
```

Output:

```
{Username: tom, Password: pass@123, Uid: U1001}
```

☐ **Note:** A map value can be any object including null.

11.4 Map Properties

Property	Description
----------	-------------

keys	Returns an iterable object representing all keys.
values	Returns an iterable object representing all values.
length	Returns the total number of key-value pairs in the Map.
isEmpty	Returns true if the Map contains no pairs.
isEmpty	Returns true if the Map contains at least one pair.

11.5 Map Functions

Function	Description
addAll()	Adds all key-value pairs from another map into this map.
clear()	Removes all key-value pairs from the map.
remove(key)	Removes the specified key and its associated value from the map.
forEach(f)	Applies function f to each key-value pair of the map.
containsKey(key)	Returns true if the map contains the given key.
containsValue(val)	Returns true if the map contains the given value.

Chapter 12: Functions — Reusable Code Blocks

Functions are the building blocks of readable, maintainable, and reusable code. A function is a named block of code that performs a specific task. Once defined, it can be called any number of times.

12.1 Defining and Calling a Function

```
// Defining a function
void greet() {
    print('Hello from a function!');
}

// Calling the function
void main() {
    greet();
}
```

Output:

```
Hello from a function!
```

12.2 Functions with Parameters

Parameters allow you to pass values into a function. The function can then use those values in its logic.

```
void printName(String name) {
    print('Hello, ${name}!');
}

void main() {
    printName('Waseem');
}
```

Output:

```
Hello, Waseem!
```

12.3 Functions with Return Values

A function can return a value back to the caller using the return keyword.

```
int add(int a, int b) {
    return a + b;
}

void main() {
    print(add(4, 5));
}
```

Output:

```
9
```

12.4 Optional Parameters

Optional parameters don't need to be passed every time a function is called.

Optional Positional Parameters — use []

```
void printInfo(String name, [int age = 0]) {  
    print('${name} is ${age} years old');  
}  
  
void main() {  
    printInfo('Ali');  
    printInfo('Sara', 22);  
}
```

Output:

```
Ali is 0 years old  
Sara is 22 years old
```

Optional Named Parameters — use { }

Named parameters must be passed by name. They are optional and can be given default values.

```
void describe({String name = 'Unknown', int age = 0}) {  
    print('${name} - age ${age}');  
}  
  
void main() {  
    describe(name: 'Ahmed', age: 25);  
    describe(age: 30);  
}
```

Output:

```
Ahmed - age 25  
Unknown - age 30
```

12.5 Recursive Functions

A recursive function calls itself. It must have a termination condition to avoid infinite loops.

```
void main() {  
    print(factorial(6));  
}  
  
factorial(number) {  
    if (number <= 0) {  
        return 1;    // termination case  
    } else {  
        return (number * factorial(number - 1));  
    }  
}
```

Output:

```
720
```

12.6 Lambda (Arrow) Functions

Lambda functions are a concise way to write single-expression functions using the => arrow syntax.

Syntax:

```
[return_type] function_name(parameters) => expression;
void main() {
    printMsg();
    print(test());
}
printMsg() => print('hello');
int test() => 123;
```

Output:

```
hello
123
```

Chapter 13: Interfaces — Contracts for Classes

An interface defines the syntax that any implementing class must follow. It specifies what methods and properties an object must have, without providing the actual implementation.

Dart does not have a special interface keyword. Instead, every class declaration acts as an implicit interface. A class uses the `implements` keyword to adopt an interface.

Note: When a class implements an interface, it **MUST** provide a concrete implementation of every method defined in that interface.

13.1 Implementing a Single Interface

Syntax:

```
class ClassName implements InterfaceName { ... }
void main() {
  ConsolePrinter cp = new ConsolePrinter();
  cp.print_data();
}

class Printer {
  void print_data() {
    print('_____Printing Data_____');
  }
}

class ConsolePrinter implements Printer {
  void print_data() {
    print('_____Printing to Console_____');
  }
}
```

Output:

```
Printing to Console
```

13.2 Implementing Multiple Interfaces

A class can implement more than one interface, separated by commas.

Syntax:

```
class ClassName implements Interface1, Interface2 { ... }
void main() {
  Calculator c = new Calculator();
  print('The gross total: ${c.ret_tot()}');
  print('Discount: ${c.ret_dis()}');
}

class Calculate_Total {
  int ret_tot() { return 0; }
}
class Calculate_Discount {
  int ret_dis() { return 0; }
}
class Calculator implements Calculate_Total, Calculate_Discount {
```

```
int ret_tot() { return 1000; }  
int ret_dis() { return 50; }  
}
```

Output:

```
The gross total: 1000  
Discount: 50
```

Chapter 14: Enumerations — Named Constants

An enumeration (enum) is used to define a set of named constant values. Instead of using raw integers or strings, enums give meaningful names to a fixed set of options — like the days of the week or the states of a process.

14.1 Declaring an Enum

```
enum enum_name {  
    value1,  
    value2,  
    value3  
}
```

Each symbol in the enumeration list represents an integer value starting from 0 by default.

14.2 Enum Example

```
enum Status {  
    none,  
    running,  
    stopped,  
    paused  
}  
  
void main() {  
    print(Status.values);  
    Status.values.forEach((v) => print('value: $v, index: ${v.index}'));  
    print('running: ${Status.running}, ${Status.running.index}');  
    print('running index: ${Status.values[1]}');  
}
```

Output:

```
[Status.none, Status.running, Status.stopped, Status.paused]  
value: Status.none, index: 0  
value: Status.running, index: 1  
value: Status.stopped, index: 2  
value: Status.paused, index: 3  
running: Status.running, 1  
running index: Status.running
```

14.3 Using Enums in switch Statements

Enums work perfectly with switch statements — each case handles one enum value.

```
void handleStatus(Status s) {  
    switch (s) {  
        case Status.running:  
            print('System is running');  
            break;  
        case Status.stopped:  
            print('System has stopped');  
            break;  
        default:
```

```
        print('Unknown state');  
    }  
}
```

Chapter 15: Runes — Unicode in Dart

Dart strings are sequences of UTF-16 code units. Unicode assigns a unique numeric value (called a code point) to every letter, digit, and symbol. A rune is an integer representing a Unicode code point.

Since Dart strings are UTF-16, 32-bit Unicode values require special handling using runes.

15.1 String.codeUnitAt() Function

Returns the 16-bit UTF-16 code unit at the specified index.

```
import 'dart:core';
void main() {
  String x = 'Runes';
  print(x.codeUnitAt(0));    // 'R' = 82
}
```

Output:

```
82
```

15.2 String.codeUnits Property

Returns an unmodifiable list of the UTF-16 code units for the entire string.

```
import 'dart:core';
void main() {
  String x = 'Runes';
  print(x.codeUnits);
}
```

Output:

```
[82, 117, 110, 101, 115]
```

15.3 String.runes Property

Returns an iterable of Unicode code points (runes) of the string. Runes extends Iterable.

```
void main() {
  'A string'.runes.forEach((int rune) {
    var character = new String.fromCharCode(rune);
    print(character);
  });
}
```

Output:

```
A
s
t
r
i
n
```

15.4 Unicode Escape Sequences

Unicode code points are usually expressed as `\uXXXX` (4 hex digits). For values outside that range, use curly brackets: `\u{XXXXXX}`.

```
void main() {  
    Runes input = new Runes(' \u{1f605} ');  
    print(new String.fromCharCode(input));    // prints emoji  
}
```

Quick Reference Summary

Concept	Key Takeaway
for loop	Definite loop — runs a fixed number of times.
for...in loop	Iterates over elements of a collection.
while loop	Indefinite — checks condition before each iteration.
do...while loop	Indefinite — always runs at least once.
break	Exits the loop immediately.
continue	Skips current iteration, jumps back to loop check.
Labels	Identifier + colon (:) to control nested loops.
String	Sequence of UTF-16 characters. Use <code>\${} </code> for interpolation.
bool	Only true or false. Dart is stricter than JavaScript.
List	Ordered collection. Index starts at 0.
Map	Key-value pairs. Keys and values can be any type.
Function	Reusable named code block. Supports optional/named params.
Lambda =>	Single-expression function shorthand.
Recursion	Function calls itself. Must have a termination condition.
Interface	Implicit in Dart — every class IS an interface.
implements	Keyword to adopt an interface. All methods must be implemented.
enum	Named constants. Values auto-indexed from 0.
Runes	Integers representing Unicode code points in a string.
codeUnitAt(i)	Returns the UTF-16 code unit at index i.
String.fromCharCode()	Creates a string from a Unicode code point.

Happy Coding with Dart!